

Pavel Ovchinnikov

Full-Stack Web Developer

BASED IN Georgia (PE)

ENGLISH B2

EDUCATION B.S. Computer Science



MY EXAMPLES



Enterprise IoT solutions

Next.js, React, MapBox, AWS, Tailwind



GraphQL boilerplate

TypeGraphQL, TypeORM, Apollo, Next.js



Human genes explorer

AWS, Next.js, React, Node.js, PostgreSQL



PWA Mobile app

Ionic, Angular, Google Maps API



Crypto payment infrastructure

Next.js, React, GraphQL, Docker

Full-stack web developer with 10+ years of experience building web and mobile applications. Comfortable across the entire stack — from database design and backend APIs to frontend UIs. Now builds software through autonomous AI agent cycles rather than writing code line by line.

When needed I use my own AI orchestrator to manage agent cycles: set a goal → execute → verify the result (evidence) → roll back on failure (rollback) → retry until the goal is achieved. I run multiple agent harness systems (opencode, Claude, and others) in parallel and compare them.

As a result, I build ERP-level web applications — the kind that usually take teams months to develop — solo, in a matter of days. Full-stack experience plus my own agent orchestration tools make that possible.

I created my own boilerplate to fast develop MVPs of any complexity quickly. Its key feature is a single source of truth for all layers (backend, frontend, database/ORM, swagger/playground).

Develop APIs, data models, and system architecture. Production experience with high-load systems built on microservices — including an SNS/SQS + Lambda pipeline processing 20k health-check requests per minute. **PostgreSQL** as the primary relational database — schema design, query optimization, indexing strategies, migrations. ORM experience with **TypeORM** and **Sequelize** across multiple production codebases. **DynamoDB** for high-performance, low-latency workloads. Builds APIs in REST (with **OpenAPI/Swagger** schemas as the contract) and **GraphQL** (schema design, resolvers, subscriptions). Production experience with multiple auth approaches: **OpenID**, **Auth0**, **AWS Amplify** (Cognito-based auth), and **next-auth** with different providers.

Experienced with **AWS** (S3, EC2, Lambda, Cognito, SNS, SQS, RDS, CloudFront) and **Google Cloud**. CI/CD pipelines, automated testing, infrastructure-as-code deploying and scaling.

Shipped production mobile apps using **React Native**, **Flutter**, or **Ionic** — native-quality experiences from a shared codebase.

Maps and geospatial integrations (**Google Maps API**, **Mapbox**, **Leaflet**) in production.

Mentors and leads developers formally and informally. Reviews teammates' code to catch bugs and keep quality high. Writes clear docs, branching policies, and linting rules. Good at detecting issues early using the terminal, curl, logs, and native database queries.

2026_{.05} - to date



Meshintex, Inc.

At Meshintex, I built a dashboard that works for any kind of IoT setup, with a backend API that takes in live sensor data. It shows real-time charts and maps with lots of markers on them. The API and dashboard are highly customizable and scalable using AI. Security was a priority the whole way through.

Kept the map view smooth with large numbers of sensors on screen, measured by lag-free pan and zoom, by implementing marker clustering. ↑ **PERFORMANCE**

Kept ingestion fast and stable under bursty sensor traffic, measured by zero dropped readings during spikes, by buffering incoming messages through an **SQS** queue ahead of processing.

↑ **PERFORMANCE**

Built role-based auth across the dashboard and API. Hardened the app against **XSS/CSRF**, and **SSRF** — sanitizing/escaping rendered input, validating request origins, and restricting outbound requests from the server. ↑ **SECURITY**

I also designed the full architecture for Lambda Digital, a crypto payment service. Each chain gets watched by its own container, which picks up transactions and sends webhooks to merchants — built with security in mind. On top of that sits a GraphQL API backed by Postgres. Everything runs in Docker and gets built and deployed automatically from one infra repo that manages all the containers.

Made each blockchain integration independently deployable and restartable, measured by one chain's outage never affecting the others, by running a dedicated watcher container per chain (**EVM, TRON, TON**, etc), orchestrated from a single infra repo with health checks and per-service memory limits. ↑ **RELIABILITY**

Ensured merchant webhooks aren't lost or duplicated under load, measured by consistent delivery during chain re-syncs, by moving webhook dispatch into a dedicated worker backed by a **BullMQ/Redis** queue instead of firing requests inline. ↑ **RELIABILITY**

Cut manual deploy steps to zero, measured by every push building and shipping on its own, by wiring up CI/CD that builds each service's Docker image and publishes it to **GHCR** for the whole container fleet. ↑ **DEV SPEED**

2022_{.07} - 2026_{.05}



Kupsilla LLC

At Kupsilla I worked on two client projects: **Strateos**, a cloud lab automation platform, and a **Genetic Science** research application.

Conducted **technical interviews** for junior and mid-level candidates, performed **code reviews**, and **mentored** developers to keep code quality high and support team growth.

Strateos – A **Cloud Lab Automation-as-a-Service** platform where users remotely run real **chemical reactions via robotic systems**.

Improved scalability and independent deployability of a large React codebase, measured by separate release cycles per team, by re-architecting the frontend into **microfrontends** using **React, TypeScript, SCSS, and Storybook**. ↑ **SCALABILITY**

Increased development speed across teams, measured by reduced duplication of UI work, by building shared UI libraries and publishing them to a private **NPM registry** used by all microfrontend modules. ↑ **DEV SPEED**

Built a complex chemical reaction constructor — a heavy custom UI component that lets users visually design lab reactions — and packaged it as a standalone NPM module to keep the main app lightweight and maintainable. ↑ **MAINTAINABILITY**

Genetic Science – A web application for searching and visualizing scientific parameters related to **human genes**.

Reduced initial page load time for data-heavy genomic pages by implementing **SSR** and enabling **gzip compression** on API responses. ↑ **PERFORMANCE**

Kept AWS infrastructure costs within the free tier, measured by zero hosting spend, by deploying on **AWS Amplify** and managing build-minute usage within free quota limits. ↓ **COST**

Built a complex interactive view combining a data plot and a collapsible rows table in one synchronized layout using **nivo.rocks**, improving data exploration for researchers. ↑ **UX**

Enabled smooth collaboration across the team, measured by a consistent CI/CD flow for all developers, by setting up **AWS Amplify** with **Amazon Cognito** (Google OAuth), linting, testing, and a **GitHub branching policy** from scratch. ↑ **TEAM FLOW**

2021_{.05} - 2022_{.07}



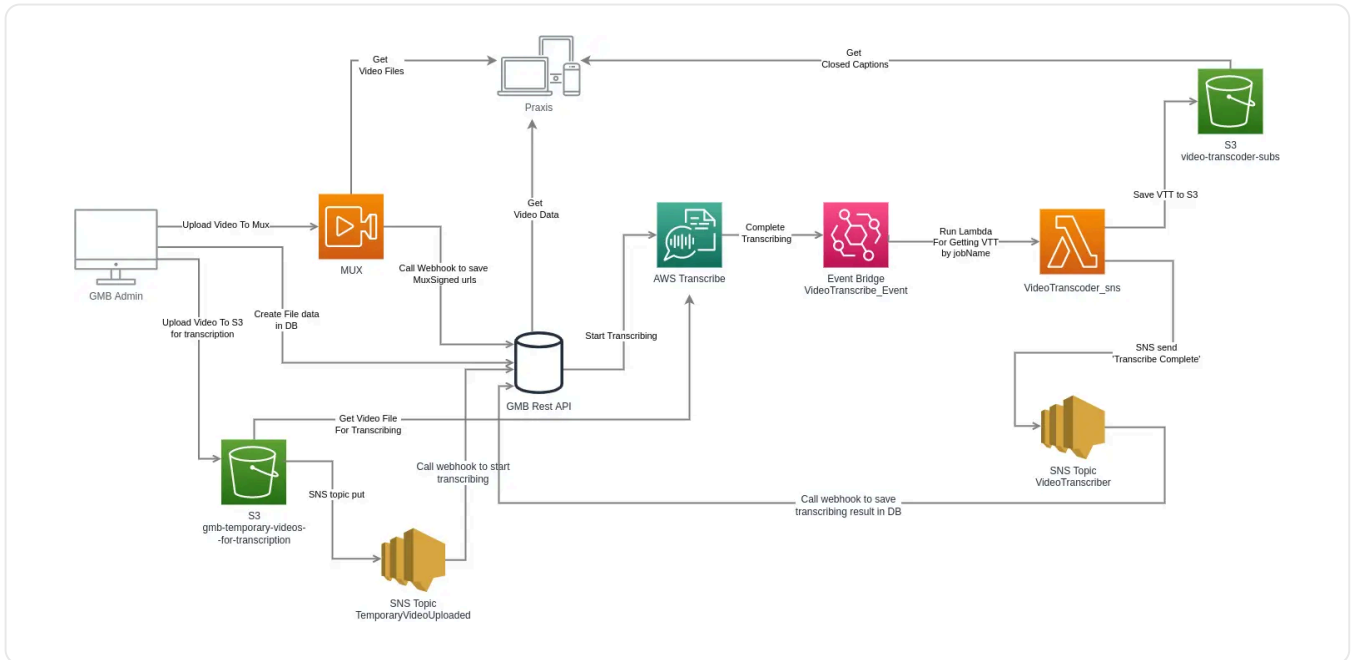
Strata K.K.

At Strata I worked on **gmb.io** and **Praxis** — a fitness video training platform for North American users. Stack: **Next.js, NestJS, PostgreSQL, TypeORM, Auth0, AWS Lambda, MUX**.

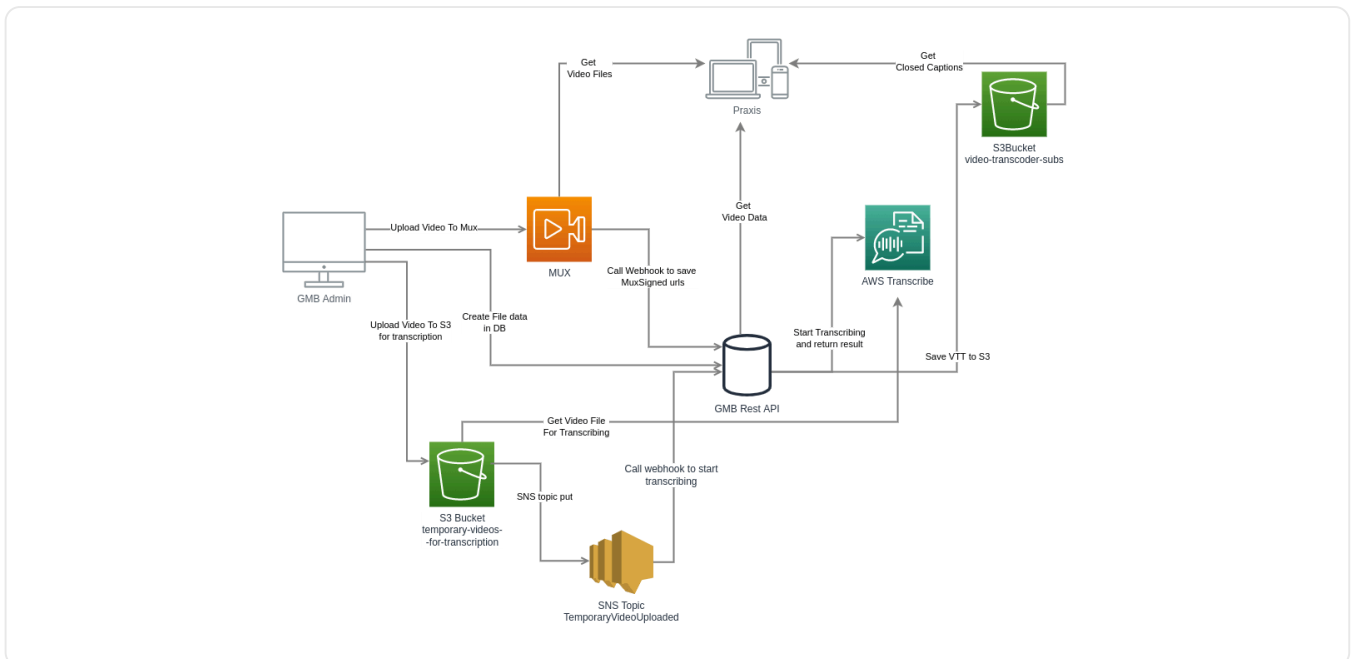
Improved video streaming quality across a wide range of devices, measured by reduced buffering and format errors, by implementing automatic platform detection that selects the optimal video codec and resolution for each client. **↑ VIDEO QUALITY**

Reduced the maintenance cost of the video upload pipeline, measured by the number of AWS services needed to operate it, by simplifying the microservice architecture and removing redundant steps. **↓ OPS COST**

PREVIOUS FLOW



SIMPLIFIED FLOW



Also maintained an admin panel built with **react-admin**.

2018.07 - 2021.04



At Nwave, an IoT company building smart parking sensors for UK clients, I worked as a Full Stack Web Developer on both frontend and backend systems managing **20,000 IoT devices**.

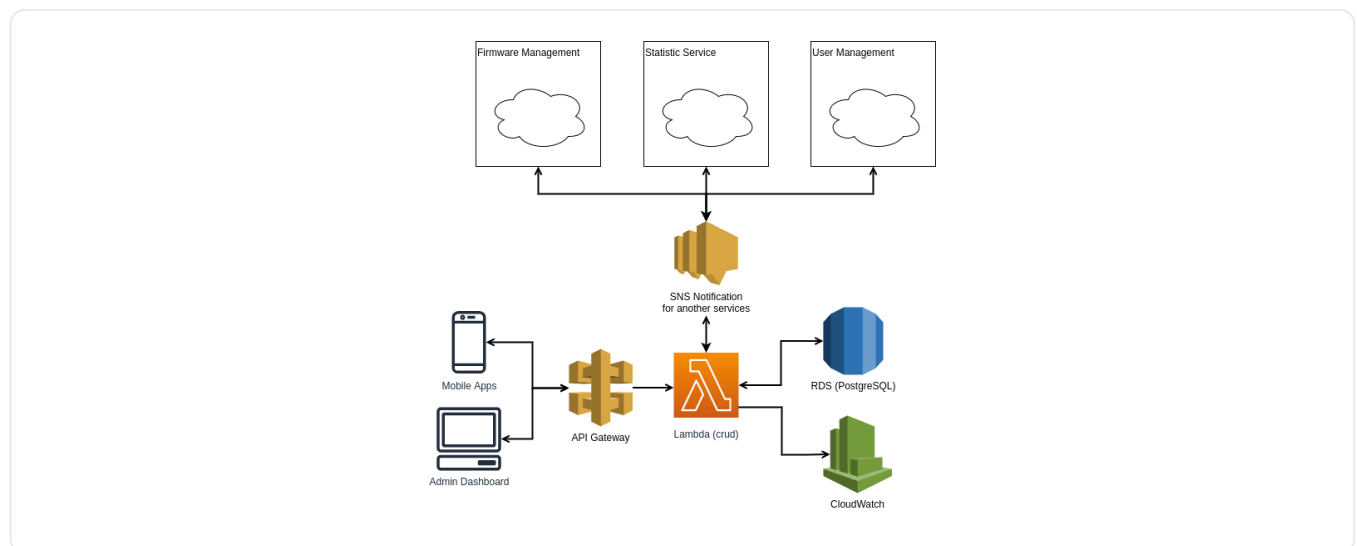
Built a **REST API** service handling full CRUD operations for 20,000 devices, achieving **99% test coverage**, by implementing it with **AWS Lambda, Node.js/TypeScript, PostgreSQL (PostGIS), AWS API Gateway**, and **AWS Cognito**, tested with **Mocha, Chai**, and the **AWS SDK**. **↑ RELIABILITY**

Improved infrastructure stability and enabled repeatable environment deployments, measured by one-command infrastructure setup, by adopting **AWS CDK** as infrastructure-as-code — replacing manual cloud configuration with version-controlled, reproducible stacks. **↑ DEVOPS**

Improved rendering performance of 20,000 map markers in the **SPlace PWA** mobile app, eliminating lag on low-end devices, by implementing marker clustering, removing unnecessary recalculations, and optimizing the **Ionic/Angular** rendering pipeline. **↑ PERFORMANCE**

Also developed and maintained an admin dashboard for monitoring sensor statuses, built with **React, Redux-Saga, TypeScript, Material-UI, AWS Amplify**, and the **Google Maps API**. The app had full unit and end-to-end test coverage, integrated with **Bitbucket CI/CD** and **Sentry**.

ARCHITECTURE DIAGRAM



2017.05 - 2018.06



At AdGuard, one of the most popular ad blockers, I contributed to both the main product and additional browser extensions, written in **JavaScript (ES6)**:

- An extension that helps users read website content hidden behind ad-blocker detection walls (not deployed, available in a [repository](#)).
- Contributions to **AdGuard Assistant** ([repository](#)).

Also worked on the front-end of the [AdGuard](#) website using **Vue**, **Vuex**, and **PostCSS**.

2015_{.05} - 2017_{.05}

Worked as a Web Developer for several companies, building and maintaining websites using **PHP**, **WordPress**, **JavaScript**, and **jQuery**. Started working with modern JavaScript frameworks, including **AngularJS** (Angular 1), and built single-page applications. Gained experience with real-time features using **WebSockets** and created interactive **SVG** animations for various projects. Automated build and deployment processes using **Grunt** and **Gulp**, which improved development consistency across environments.